Stat 418: Tools in Data Science

Course Description

Stats 418: Tools in Data Science is a graduate level statistics course restricted to UCLA Masters in Applied Statistics students. The course will present current tools for data acquisition, transformation and analysis, data visualization, and machine learning and tools for reproducible data analysis, collaboration, and model deployment used by data scientists in practice. Advanced R packages and Python libraries, analytical databases, high-performance machine learning libraries, big data tools.

MASDS Departmental Disclaimers:

For students trying to take or audit from outside the MASDS program.

Taking or auditing 400 courses is simply is not permitted because this is a self-supporting program. Sorry, unfortunately, you will NOT be able to take any of the 400 level Stats courses.

There are NO exceptions that can be made by the department. These classes were designed specifically for students who applied directly to the program.

The students of this program are also not allowed to audit or enroll in classes outside of the program as it was created for working professionals.

If you would like to apply for the program, you are welcome to do so:
https://master.stat.ucla.edu/admissions/

Information is found here: https://master.stat.ucla.edu/ And here: https://master.stat.ucla.edu/faq/

Course Pre-reqs

- Stat 404: Statistical Computing
- Stat 405: Data Management

Welcome to STAT 418

- Building Real, Deployable Data Products
- This course focuses on the practical tools that data scientists use to build real, deployable data products. Over 10 weeks, we'll take a data science project from initial idea all the way through to a deployed application that others can use.
- 10 weeks: Idea → Deployed Application
- Hands-on learning with industry tools
- Tools change, fundamentals endure
- The landscape of data science tools changes rapidly. What was cutting edge five years ago might be obsolete today. But the fundamental concepts - version control, reproducible environments, testing, deployment - these endure.

Nate Langholz, Ph.D.

Statistician - Data Scientist

Hengzhi He

Stat PhD working on
Generative AI

Class Intros…

Class Intros

- Name
- Background/Current Job
- Why this class? Why this program?
- Anything in particular you’d like to learn?

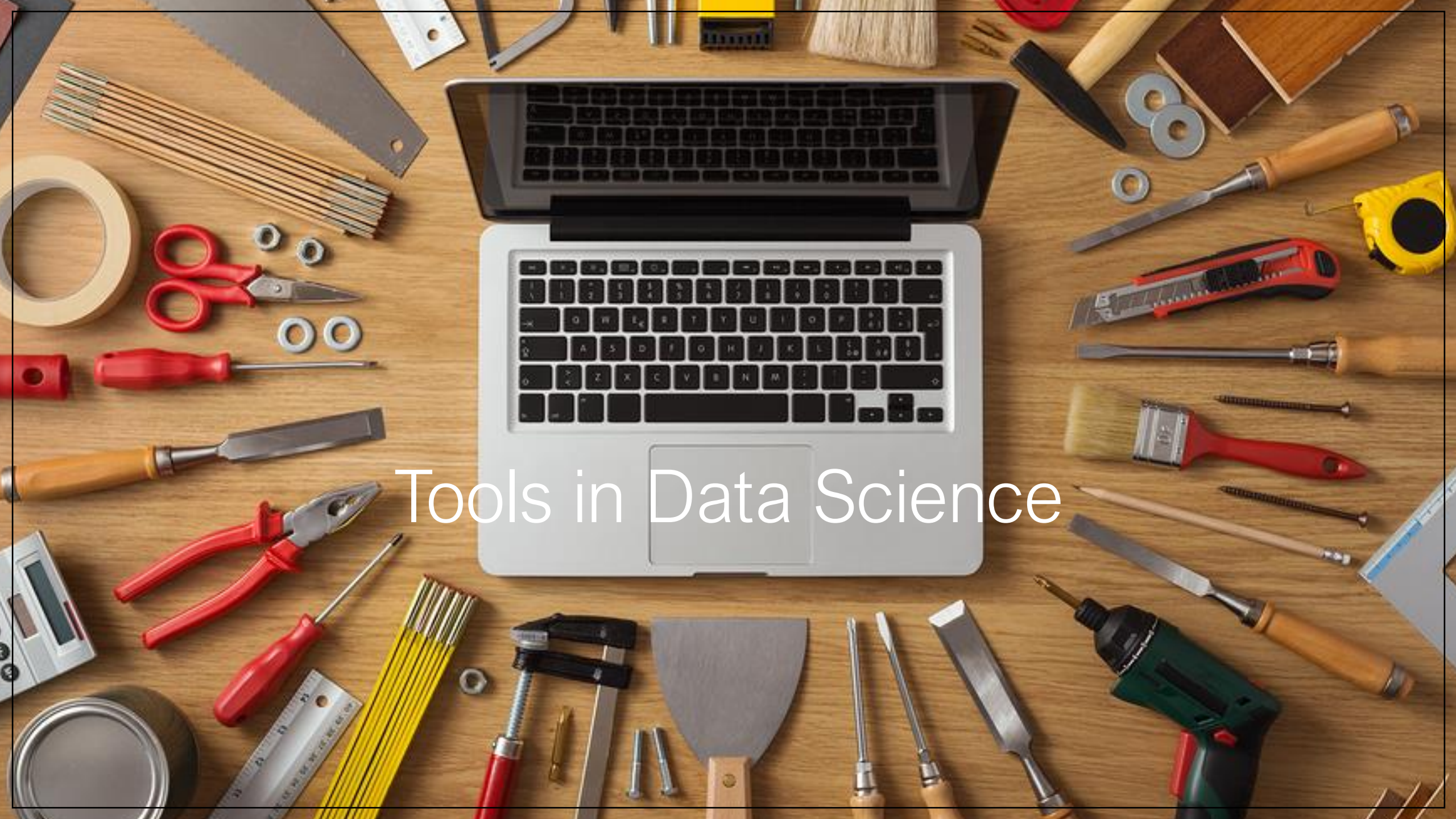
Course Info and Logistics

https://github.com/natelangholz/stat418-tools-in-datascience-2026

or ask on Slack…
#all-ucla-stat-418-tools-ds-2026

Course Structure

- Progressive Learning Over 10 Weeks
  - Tuesdays 6:00-8:50 PM
  - 4 problem sets on alternating weeks
  - Individual final project throughout the quarter
  - Week 6: Proposal presentation with acquired data
  - Week 10: Final presentation of completed project
- Course Arc:
  - Weeks 1-3: Foundation (Git, Python, Command Line, Containers)
  - Weeks 4-5: Data Acquisition & AI Integration (Web Scraping, LLMs, RAG)
  - Weeks 6-8: Deployment & Applications (Streamlit, APIs, MCP Servers)
  - Weeks 9-10: Production Systems & Presentations (CI/CD, Final Projects)



Tools in Data Science

Week 1: Introduction & Modern Development Environment

AI-Assisted Development in 2026

- AI Assistants as Productivity Tools
- AI coding assistants are everywhere in 2026. They can write code, explain code, debug code, and generate documentation. This fundamentally changes how we approach software development, but it doesn't make learning to code obsolete.

The philosophy:

  - AI enhances productivity, doesn't replace understanding
  - Like calculators in math: frees you for higher-level thinking
  - Learn when to trust, when to question
  - Focus on architecture and problem-solving, not syntax memorization

> **Tao:** These problems are like distant locations that you would hike to. And in the past, you would have to go on a journey. You can lay down trail markers that other people could follow, and you could make maps.
>
> AI tools are like taking a helicopter to drop you off at the site. You miss all the benefits of the journey itself. You just get right to the destination, which actually was only just a part of the value of solving these problems.

AI Coding Assistants

- The AI Revolution in Software Development (2021-2026)
- The rise of AI coding assistants represents a fundamental shift in how we write software:
- Timeline:
  - 2021: GitHub Copilot launches (first mainstream AI assistant, paid)
  - 2022-2023: Explosion of alternatives from multiple companies
  - 2024-2025: Free tiers emerge, open source models become available
  - 2026: Standard practice in professional development

Copilot

CURSOR

What AI Assistants Excel At

AI assistants are particularly good at certain types of coding tasks:

- Code Generation:
  - Boilerplate code - repetitive, standard code every project needs
  - Syntax help - when you know what to do but can't remember exact syntax
  - Function scaffolding - creating function signatures and basic structure

Code Understanding:

  - Explaining unfamiliar code - especially useful with new libraries
  - Documentation generation - docstrings, comments, README files
  - Code translation - converting between languages or formats

Development Tasks:

  - Unit test generation - creating test cases for your functions
  - Debugging simple errors - explaining what went wrong
  - Code improvements - suggesting more efficient or readable approaches
  - Refactoring - restructuring code while maintaining functionality

Deep learning multiple aspects of coding is dead?

I do think there will continue to have value understanding how code works.

But how deep of knowledge is needed?

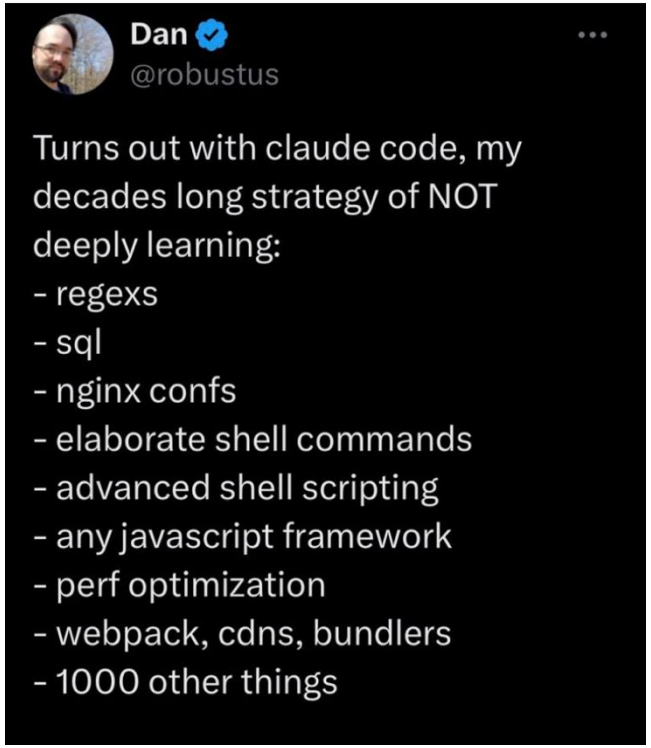
Knowing how to prompt AI comes from understanding coding patterns,

But being specialized in all aspects of coding knowledge is not only not fully feasible,

It is becoming less necessary.

However, being an expert will still has its place.

It's not about having all the answers, but knowing how to find them.



What AI Assistants Cannot Do

AI assistants have important limitations that mean you're still responsible for the critical aspects of development:

Context Limitations:

- Don't understand your full system - only see code you show them
- Can't make architectural decisions - don't understand long-term implications
- Don't know your requirements - can only work with what you tell them

Technical Limitations:

- Struggle with domain-specific logic - especially in specialized fields
- Can't debug complex interactions - between multiple systems
- Don't understand security implications - or performance at scale
- May suggest outdated approaches - training data has cutoff dates
- Can hallucinate - generate plausible but incorrect code

Your Ongoing Responsibilities:

- Understanding the generated code - don't just copy and paste
- Testing thoroughly - AI can't verify correctness
- Making architectural decisions - overall system design
- Applying domain knowledge - understanding what makes sense in your field
- Security review - ensuring code is safe and secure

The Modern Development Workflow

The workflow for building data science projects has evolved with AI assistants, but humans still drive the critical decisions:

AI-Assisted Development Process:

1. Define the problem (human) - understand requirements and constraints
2. Research approaches (human + AI) - explore solutions and best practices
3. Design architecture (human) - make high-level structural decisions
4. Generate boilerplate (AI) - create standard code structures
5. Implement domain logic (human + AI) - add business-specific functionality
6. Review and refine (human) - ensure quality and correctness
7. Test thoroughly (human + AI) - verify functionality and edge cases
8. Document (human + AI) - explain what was built and why
9. Deploy and monitor (human + AI) - put into production

VSCode - The Modern Code Editor

- Evolution of Code Editors
- The history of code editors shows a progression toward more integrated, user-friendly tools:
- 1970s-1980s: vi, Emacs (terminal-based, steep learning curve)
- 1990s-2000s: Eclipse, Visual Studio (heavy IDEs, slow, language-specific)
- 2010s: Sublime Text, Atom (lightweight, fast, but limited features)
- 2015-present: VSCode
  - Free and open source (Microsoft)
  - Fast and lightweight (built on Electron)
  - Massive extension ecosystem (JavaScript-based extensions)
  - Cross-platform (Mac, Windows, Linux)
  - Strikes perfect balance between lightweight and feature-rich

Why VSCode for Data Science

Integrated Workflow - Everything in One Place

VSCode offers a complete development environment without the complexity of traditional IDEs:

Core Features:

- Code editor with syntax highlighting and IntelliSense
- Integrated terminal - run commands without leaving the editor
- Git interface - visual diff, staging, commits, push/pull
- Jupyter notebook support - run notebooks directly in VSCode
- Debugger - step through code and inspect variables
- Extension ecosystem - customize for your specific needs

No Context Switching:

- Write code, run commands, commit to Git, use AI assistants - all without leaving VSCode. This integrated workflow dramatically improves productivity.

Python-Specific Benefits:

- IntelliSense (intelligent code completion)
- Linting (automatic code quality checks)
- Formatting (automatic code style)
- Testing integration
- Virtual environment detection

Essential VSCode Extensions

- Three Must-Have Extensions for This Course

1. Python IntelliSense, debugging, linting, formatting
   - Virtual environment integration
   - Jupyter notebook support
   - Testing framework integration
2. Open source AI coding assistant for VSCode
   - Context-aware suggestions based on your codebase
   - Code generation, explanation, debugging help
   - Documentation writing assistance
3. GitLens (Enhanced Git Integration)
   - Visual Git history and blame annotations
   - See who changed each line of code and when
   - Branch comparison and visualization
   - Commit history exploration
   - Essential for understanding project evolution

The Python Package Management Problem

- Python has become the dominant language for data science, but it has a fundamental problem:
  - `Project A needs:` `Project B needs:`
  - `- pandas 1.5.0` `- pandas 2.0.0`
  - `- numpy 1.23.0` `- numpy 1.24.0`
  - `But system Python can only have ONE version of each package!`
- If you install pandas 2.0.0 for Project B, you break Project A.
- Solution: Virtual Environments
  - A virtual environment is an isolated Python installation for a specific project:
  - Each project gets its own environment with its own packages
  - Project A's environment has pandas 1.5.0
  - Project B's environment has pandas 2.0.0
  - They don't interfere with each other
  - Full reproducibility when sharing projects

The Python Package Management Saga

- Python's ecosystem has had many competing tools for managing packages and environments, and they've all had significant problems:
  - virtualenv (2007): Virtual environments, but clunky workflow
  - pip (2008): Package installer, but slow dependency resolution
  - conda (2012): Environments + packages, but heavy and slow
  - poetry (2018): Modern dependency management, but complex
  - pipenv (2017): Attempted to unify pip + virtualenv, but slow and confusing

The hardest part of Python is managing Python itself

Enter uv: Modern Python Package Management

- Released 2024 by Astral (creators of Ruff)
- uv represents a new approach to Python package management, designed from the ground up to be fast, simple, and reliable:
- Key Advantages:
  - Written in Rust: 10-100x faster than pip
  - Unified tool: Handles both packages AND environments
  - Simple, intuitive commands: One way to do things
  - Compatible: Works with pip and requirements.txt
  - Cross-platform: Same experience on Mac, Windows, Linux
  - Free and open source
- Why uv for this course:
  - Fast enough you won't wait for installations
  - Simple enough you won't get confused
  - Modern and actively developed
  - Solves practical problems that have plagued Python for years

uv Workflow

- Creating a Project
  - `mkdir my-project`
  - `cd my-project`
  - `uv init`
  - `uv venv`
  - `source .venv/bin/activate  # Mac/Linux`
  - `.venv\Scripts\activate  # Windows`
- Installing Packages
  - `uv add pandas numpy matplotlib`
  - `uv pip install pandas numpy matplotlib`
  - `uv pip install -r requirements.txt`
  - `uv pip freeze > requirements.txt`

- When you create a virtual environment, uv creates a .venv directory containing a complete Python installation. When activated, your shell uses this isolated Python instead of your system Python.

Version Control with Git

If you've ever worked on a document and ended up with files named:

- `final_version.doc`
- `final_version_2.doc`
- `final_version_ACTUALLY_FINAL.doc`

You understand the problem version control solves. Now imagine that problem with code, where you have dozens of files that all depend on each other, and multiple people making changes simultaneously.

What Version Control Provides:

- Track every change made to your code over time
- See who changed what, when, and why
- Experiment without fear of breaking working code
- Collaborate without conflicts
- Roll back to any previous state if something goes wrong

How Git Works

- Three States of Your Code
  - 1. Working Directory - Your actual files that you're editing
  - 2. Staging Area (Index) - Files you've marked as ready to save
  - 3. Repository (.git directory) - The committed history of your project

- Basic Workflow:
  - `git status` # Check what's changed
  - `git add filename.py` # Stage specific changes
  - `git add .` # Stage all changes
  - `git commit -m "Add feature"` # Commit with descriptive message
  - `git push origin main` # Push to GitHub

- Make changes → Stage changes → Commit changes → Push to remote

effective communication skills

critical skill for data scientist requiring practice and feedback to develop

marketing

Why do public work?